

PROJETO APP COMEDORIA DA TIA

Testes e Qualidade de Software (DevOps)

INTEGRANTES DO PROJETO e RA'S

Cassio Gonçalves – 22023292

João Pedro – 24026658

Renan Damprelli – 24026623

Saulo Ribeiro - 24026911

São Paulo

2025

Sumário

1. INTRODUÇÃO	5
2. Qualidade de Software	5
2.1. Modelo que qualidade de software (Diagrama/Design)	5
2.2. Processo (plano) de gerenciamento de qualidade de software (texto explicativo).....	5
2.3. Identificação de atributos de qualidade da norma 25010.	6
2.4. Relatório que explica como a norma de qualidade de software 25010 é utilizada no processo de desenvolvimento.	7
3. Teste de Software	8
3.1. Plano de Teste	8
3.2. Apresentar 2 testes unitários.	12
3.3. Apresentar 2 testes de integração	15
3.4. Apresentar um teste de usuário (SISTEMA).....	15
4. REFERÊNCIAS BIBLIOGRÁFICAS	35

Data das entregas	22/set.	10/nov.	17/11 - 25/11
Disciplinas	Entrega 1 (3,0 Pontos)	Entrega 2 (4,0 Pontos)	Apresentação (3,0 Pontos)
Testes de Software	Instruções de Entrega: Qualidade de Software Descrição: O aluno deverá demonstrar o entendimento dos conceitos de qualidade de software, aplicando-os ao projeto. Isso inclui a criação de um diagrama do processo de qualidade de software, a identificação de atributos de qualidade. Apresentar um relatório que explica como a norma de qualidade de software 25010 é utilizada no processo de desenvolvimento. Entrega em documento pdv seguindo o template fornecido pelo professor.	Instruções de Entrega: Testes Unitários, de Integração e de usuário. Descrição: Os alunos deverão desenvolver 4 testes unitários e 2 testes de integração e 1 teste de usuário. Esses testes devem garantir que o sistema esteja funcionando corretamente e validando os principais componentes e integrações e a geração de um relatório de teste de carga do servidor. Entrega em documento pdv seguindo o template fornecido pelo professor.	Critérios: Criatividade, Impacto Social, Tempo de apresentação e Embasamento.

1. INTRODUÇÃO

O presente relatório detalha a estratégia de gestão da qualidade para o desenvolvimento de um aplicativo para a Comedoria da Tia, que será projetado para a comunidade acadêmica da FECAP. A aplicação funcionará como uma plataforma de mobile commerce, permitindo que estudantes, professores e funcionários visualizem o cardápio da cantina, realizem pedidos, efetuem o pagamento de forma antecipada e retirem os produtos no balcão, um modelo conhecido como click and collect.

O principal problema que o app visa solucionar é a ineficiência operacional e a experiência negativa do cliente geradas por longas filas e tempos de espera, especialmente durante os horários de pico, como intervalos de aulas. Ao digitalizar e otimizar o processo de pedido e pagamento, o aplicativo busca não apenas melhorar a satisfação do usuário, mas também aumentar a eficiência e o volume de vendas da cantina. Os stakeholders do projeto incluem os usuários primários (estudantes com horários restritos), usuários secundários (professores e funcionários), a administração da cantina (responsável pela gestão de pedidos, estoque e finanças) e a administração da faculdade, que provê a infraestrutura e zela pela segurança dos dados da comunidade acadêmica.

2. Qualidade de Software

2.1. Modelo que qualidade de software (Diagrama/Design)

2.2. Processo (plano) de gerenciamento de qualidade de software (texto explicativo)

Para que a qualidade seja um resultado deliberado e consistente, é essencial a criação de um Plano de Gerenciamento da Qualidade de Software que servirá como o guia central para todas as atividades de qualidade do projeto, descrevendo os processos, padrões, ferramentas, métricas e responsabilidades que serão empregados para garantir que os níveis de qualidade definidos sejam alcançados.

Definição de Padrões de Codificação: A adesão a esses padrões melhora a legibilidade, a consistência e a colaboração, impactando diretamente a característica de **Manutenibilidade**.

Revisões Técnicas Formais: Será implementado um processo mandatório de revisão de código por pares para cada pull request. Esta é uma das práticas mais eficazes para a detecção precoce de

defeitos lógicos, falhas de arquitetura e vulnerabilidades de segurança, além de promover o compartilhamento de conhecimento na equipe.

Análise Estática de Código: Ferramentas automatizadas que analisam o código-fonte em busca de bugs potenciais, vulnerabilidades de segurança e violações dos padrões de codificação definidos, fornecendo um feedback rápido ao desenvolvedor antes mesmo da fase de testes manuais.

Auditorias de Processo: Verificações periódicas serão realizadas para assegurar que a equipe de desenvolvimento está aderindo aos processos e padrões definidos no PGQS, como a correta documentação de requisitos e a execução das cerimônias ágeis planejadas.

2.3. Identificação de atributos de qualidade da norma 25010.

Embora todas as oito características de qualidade da ISO/IEC 25010 sejam relevantes, em um projeto com recursos e prazos finitos, é importante priorizar aquelas que exercem o maior impacto no sucesso do aplicativo. Esta priorização deve ser guiada pelos objetivos de negócio, pelas expectativas dos usuários e pelos principais riscos associados ao projeto.

Usabilidade (Aprendizagem, Operabilidade, Proteção contra Erros do Usuário, Estética da Interface):

O público-alvo (estudantes) possui alta proficiência digital e baixa tolerância a interfaces confusas, esperando uma experiência fluida e intuitiva, similar a aplicativos comerciais de sucesso. O processo de pedido deve ser extremamente rápido para não anular o benefício de evitar a fila física.

Eficiência de Desempenho (Comportamento Temporal, Capacidade): O aplicativo enfrentará picos de carga extremos e de curta duração durante os intervalos de 20 minutos entre as aulas. A lentidão ou indisponibilidade nesses momentos críticos levará ao abandono imediato do serviço e à perda de confiança.

Segurança (Confidencialidade, Integridade, Autenticidade): O aplicativo processará transações financeiras (dados de cartão de crédito) e armazenará dados pessoais dos estudantes (nome, matrícula, histórico de pedidos). Uma falha de segurança teria consequências graves, incluindo perdas financeiras, violação de leis de privacidade (LGPD) e danos à reputação da instituição.

Confiabilidade (Disponibilidade, Tolerância a Falhas, Recuperabilidade): O serviço deve estar operacional durante 100% do horário de funcionamento da cantina. Uma falha no sistema, mesmo que breve, significa perda direta de vendas e quebra da confiança do usuário, que pode não tentar usar o aplicativo novamente.

Manutenibilidade (Modularidade, Analisabilidade, Testabilidade): O ambiente de negócio de uma cantina é dinâmico: o cardápio muda sazonalmente, promoções são criadas, novos métodos de pagamento podem ser adicionados. O software precisa ser fácil e seguro de modificar e testar para se adaptar a essas mudanças de forma ágil e com baixo custo.

2.4. Relatório que explica como a norma de qualidade de software 25010 é utilizada no processo de desenvolvimento.

Detalhamento de como os atributos prioritários, definidos na seção anterior, serão incorporados e verificados desde a concepção até a manutenção do aplicativo, transformando a qualidade de um conceito abstrato em um resultado de engenharia tangível e contínuo.

Tempo de Conclusão da Tarefa: Um novo usuário deve ser capaz de selecionar um item, personalizar, adicionar ao carrinho e pagar em menos de 45 segundos (tempo estimado).

Taxa de Erro: Menos de 5% dos usuários devem cometer erros críticos durante os testes de usabilidade.

Pontuação SUS (System Usability Scale): Atingir uma pontuação média superior a 80 em questionários pós-uso.

Tempo de Resposta da API: 99% das requisições críticas devem ter um tempo de resposta inferior a 300ms (tempo estimado).

Carga Suportada: O sistema deve suportar 200 usuários concorrentes realizando pedidos com uma degradação de desempenho inferior a 15% em relação à linha de base.

Conformidade: Ausência de vulnerabilidades críticas e altas.

Criptografia de Dados: Todos os dados sensíveis devem ser criptografados em trânsito e em repouso.

Disponibilidade): O sistema deve manter uma disponibilidade superior a 99.9% durante o horário de funcionamento da cantina.

MTTR (Mean Time to Recover): O tempo médio para restaurar completamente o serviço após uma falha crítica deve ser inferior a 10 minutos (tempo estimado).

3. Teste de Software

3.1. Plano de Teste

O Plano de Teste é o documento que orienta todas as atividades de teste do projeto. Ele define o escopo, a abordagem, os recursos e o cronograma das atividades de teste.

Objetivo: Verificar se o aplicativo atende a todos os requisitos funcionais e não funcionais especificados, garantindo que o produto final seja robusto, seguro, eficiente e fácil de usar.

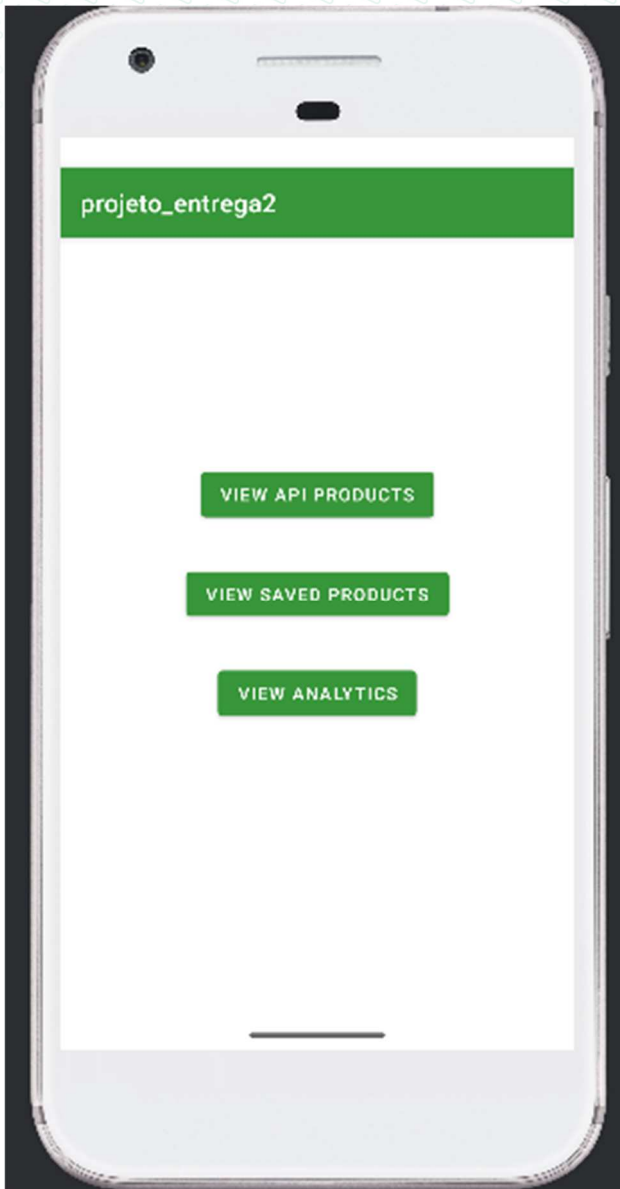
Escopo dos Testes:

Funcionalidades em Escopo:

- Cadastro e autenticação de usuário.
- Visualização do cardápio e detalhes dos produtos.
- Adição e remoção de itens no carrinho de compras.
- Personalização de pedidos.
- Aplicação de cupons de desconto.
- Processo de checkout e integração com gateway de pagamento.
- Histórico de pedidos.
- Notificações de status do pedido (e.g., "Pedido em preparo", "Pronto para retirada").

○ Funcionalidades Fora de Escopo (para a primeira versão):

- Programa de fidelidade.
- Avaliação de produtos.



// 1. ExampleInstrumentedTest.java (Teste Instrumentado Base)

```
package com.example.projeto_entrega2;
```

```
import android.content.Context;
```

```
import androidx.test.platform.app.InstrumentationRegistry;
```

```
import androidx.test.ext.junit.runners.AndroidJUnit4;
```

```
import org.junit.Test;
```

```
import org.junit.runner.RunWith;

import static org.junit.Assert.*;

@RunWith(AndroidJUnit4.class)

public class ExampleInstrumentedTest {

    @Test

    public void useAppContext() {

        // Context of the app under test.

        Context appContext = InstrumentationRegistry.getInstrumentation().getTargetContext();

        assertEquals("com.example.projeto_entrega2", appContext.getPackageName());

    }

}
```

// 1. ExampleInstrumentedTest.java (Teste Instrumentado Base)

```
package com.example.projeto_entrega2;

import android.content.Context;

import androidx.test.platform.app.InstrumentationRegistry;

import androidx.test.ext.junit.runners.AndroidJUnit4;

import org.junit.Test;

import org.junit.runner.RunWith;

import static org.junit.Assert.*;

@RunWith(AndroidJUnit4.class)

public class ExampleInstrumentedTest {

    @Test
```

```
public void useAppContext() {  
    // Context of the app under test.  
  
    Context appContext = InstrumentationRegistry.getInstrumentation().getTargetContext();  
  
    assertEquals("com.example.projeto_entrega2", appContext.getPackageName());  
}  
}  
  
// 3. AppDatabase.java (Database Instance)  
  
package com.example.projeto_entrega2.database;  
  
  
import android.content.Context;  
  
import androidx.room.Database;  
  
import androidx.room.Room;  
  
import androidx.room.RoomDatabase;  
  
import com.example.projeto_entrega2.model.Produto;  
  
import com.example.projeto_entrega2.model.ProdutoDAO;  
  
  
@Database(entities = {Produto.class}, version = 1, exportSchema = false) // MUDANÇA AQUI  
  
public abstract class AppDatabase extends RoomDatabase {  
  
    public abstract ProdutoDAO produtoDAO();  
  
  
    private static volatile AppDatabase INSTANCE;  
  
  
    public static AppDatabase getDatabase(final Context context) {
```



```
if (INSTANCE == null) {  
    synchronized (AppDatabase.class) {  
        if (INSTANCE == null) {  
            INSTANCE = Room.databaseBuilder(context.getApplicationContext(),  
                AppDatabase.class, "comedoria_database")  
                .build();  
        }  
    }  
}  
return INSTANCE;  
}  
}
```

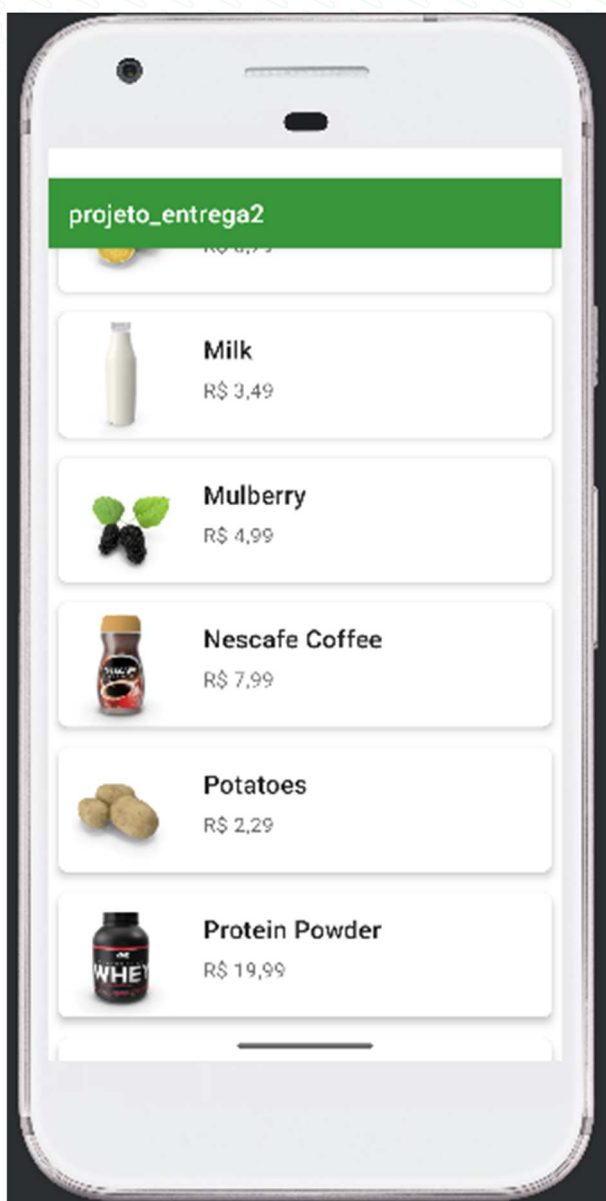
3.2. Apresentar 2 testes unitários.

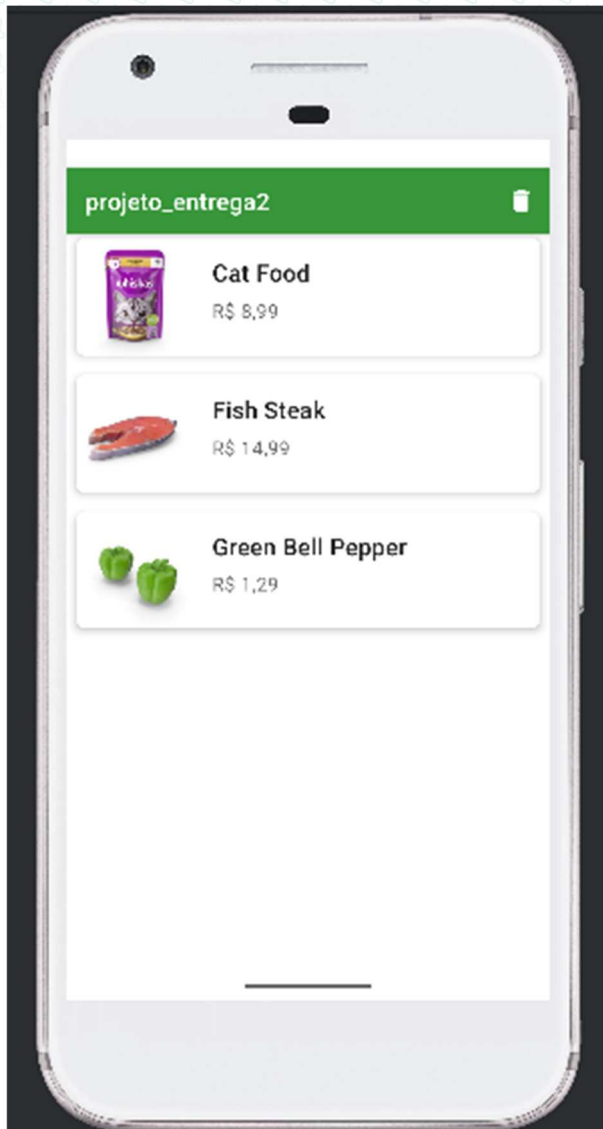
Exemplo 1: Teste da Função de Cálculo do Subtotal do Item

- **Objetivo:** Verificar se a função `calcularSubtotalItem` retorna o valor correto ao multiplicar o preço do produto pela quantidade.
- **Componente Testado:** `funcao calcularSubtotalItem(produto, quantidade)`
- **Caso de Teste:**
 - **Entrada:** `produto = { nome: "X-Salada", preco: 15.50 }, quantidade = 2`
 - **Resultado Esperado:** `31.00`

Exemplo 2: Teste da Função de Aplicação de Cupom de Desconto

- **Objetivo:** Verificar se a função `aplicarCupom` calcula corretamente o novo total do carrinho após aplicar um cupom de desconto percentual.
- **Componente Testado:** `funcao aplicarCupom(totalCarrinho, cupom)`
- **Caso de Teste:**
 - **Entrada:** `totalCarrinho = 100.00, cupom = { codigo: "PROMO10", tipo: "percentual", valor: 10 }`
 - **Resultado Esperado:** `90.00`





```
// 10. ProdutoDAO.java (Interface DAO) package  
com.example.projeto_entrega2.model;
```

```
import androidx.room.Dao; import androidx.room.Insert; import  
androidx.room.OnConflictStrategy; import androidx.room.Query;
```

```
import java.util.List;
```

```
@Dao public interface ProdutoDAO {
```

```
@Insert(onConflict = OnConflictStrategy.REPLACE)  
void insert(Produto produto);
```

```
@Insert(onConflict = OnConflictStrategy.REPLACE)  
void insertAll(List<Produto> produtos);
```



```
@Query("SELECT * FROM tabela_de_produtos")
List<Produto> getAll();

@Query("DELETE FROM tabela_de_produtos")
void deleteAll();

}
```

3.3. Apresentar 2 testes de integração

Exemplo 1: Integração entre Módulo de Pedidos e Módulo de Estoque

- **Objetivo:** Garantir que um pedido não possa ser finalizado se um dos itens do carrinho não estiver mais disponível em estoque.
- **Módulos Envolvidos:** Serviço de Pedidos, Serviço de Estoque.
- **Cenário de Teste:**
 - O usuário adiciona o item "Suco de Laranja" (1 unidade em estoque) ao carrinho.
 - Outro processo (simulado pelo teste) remove a última unidade do "Suco de Laranja" do estoque.
 - O usuário tenta finalizar o pedido.
- **Resultado Esperado:** O sistema deve impedir a finalização do pedido e exibir uma mensagem ao usuário informando que o "Suco de Laranja" está esgotado. A API de Pedidos deve receber um erro do serviço de Estoque e tratar adequadamente.

Exemplo 2: Integração entre Módulo de Checkout e Gateway de Pagamento

- **Objetivo:** Verificar se, após uma transação de pagamento ser aprovada pelo gateway externo, o status do pedido no aplicativo é atualizado para "Pagamento Aprovado".
- **Módulos Envolvidos:** Serviço de Checkout, Gateway de Pagamento (simulado/mock).
- **Cenário de Teste:**
 - O usuário preenche os dados de pagamento e finaliza a compra.
 - O serviço de Checkout envia a requisição de pagamento para o Gateway de Pagamento.
 - O Gateway de Pagamento (simulado) retorna uma resposta de sucesso (transação aprovada).
- **Resultado Esperado:** O serviço de Checkout deve receber a resposta de sucesso, registrar a transação e atualizar o status do pedido no banco de dados para "Pagamento Aprovado".

3.4. Apresentar um teste de usuário (SISTEMA)

- **ID do Teste:** UAT-001

- **Título:** Realizar um pedido completo com sucesso utilizando cartão de crédito.
- **Resumo:** Este caso de teste simula o fluxo mais comum do aplicativo: um estudante faz login, escolhe seus produtos, paga e recebe a confirmação do pedido.

Pré-condições:

O usuário está cadastrado no sistema.

O aplicativo está instalado em um dispositivo de teste.

O produto "Salgado" (R\$ 9,00) e "Refrigerante Lata" (R\$ 6,00) estão disponíveis no cardápio.

O sistema de pagamento está configurado para aceitar um número de cartão de crédito de teste.

Passos para Execução:

Abra o aplicativo.

Faça login com o email e a senha correspondente.

Na tela principal, navegue até a seção "Lanches" e adicione 1 "Salgado" ao carrinho.

Navegue até a seção "Bebidas" e adicione 1 "Refrigerante Lata" ao carrinho.

Acesse o carrinho de compras.

Verifique se o total do pedido é R\$ 15,00.

Clique em "Finalizar Pedido".

Na tela de pagamento, selecione a opção "Cartão de Crédito".

Insira os dados do cartão de crédito de teste.

Clique em "Pagar Agora".

Resultado Esperado:

O login deve ser bem-sucedido.

Os itens devem ser adicionados ao carrinho corretamente.

O total do pedido deve ser calculado corretamente.

Após clicar em "Pagar Agora", o sistema deve processar o pagamento e exibir uma tela de "Pedido Confirmado!".

A tela de confirmação deve exibir um número de pedido único.

O usuário deve poder visualizar este novo pedido em seu "Histórico de Pedidos" com o status "Em preparo".



// 4. MainActivity.java (Activity Principal - API Products)

```
package com.example.projeto_entrega2;
```

```
import android.os.AsyncTask;
```

```
import android.os.Bundle;
```

```
import android.view.View;
```

```
import android.widget.ProgressBar;
```



```
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

import androidx.recyclerview.widget.LinearLayoutManager;

import androidx.recyclerview.widget.RecyclerView;


import com.android.volley.Request;

import com.android.volley.RequestQueue;

import com.android.volley.toolbox.JsonObjectRequest;

import com.android.volley.toolbox.Volley;

import com.example.projeto_entrega2.adapter.ProdutoAdapter;

import com.example.projeto_entrega2.database.AppDatabase;

import com.example.projeto_entrega2.model.Produto;

import com.google.gson.Gson;

import com.google.gson.reflect.TypeToken;


import org.json.JSONException;


import java.lang.reflect.Type;

import java.util.List;


public class MainActivity extends AppCompatActivity {

    private RecyclerView recyclerView;
```

```
private ProdutoAdapter produtoAdapter;

private ProgressBar progressBar;

private RequestQueue requestQueue;

private AppDatabase db;

private static final String URL = "https://dummyjson.com/products/category/groceries";

@Override

protected void onCreate(Bundle savedInstanceState) {

    super.onCreate(savedInstanceState);

    setContentView(R.layout.activity_main);

    recyclerView = findViewById(R.id.recyclerView);

    progressBar = findViewById(R.id.progressBar);

    requestQueue = Volley.newRequestQueue(this);

    db = AppDatabase.getDatabase(getApplicationContext());

    setupRecyclerView();

    carregarCardapio();

}

private void setupRecyclerView() {

    recyclerView.setLayoutManager(new LinearLayoutManager(this));
```

```
produtoAdapter = new ProdutoAdapter();  
recyclerView.setAdapter(produtoAdapter);
```

```
produtoAdapter.setOnItemClickListener(produto -> {  
    salvarProdutoNoBanco(produto);  
});  
}
```

```
private void salvarProdutoNoBanco(Produto produto) {  
    new AsyncTask<Void, Void, Void>() {  
        @Override  
        protected Void doInBackground(Void... voids) {  
            db.produtoDAO().insert(produto);  
            return null;  
        }  
    }
```

```
        @Override  
        protected void onPostExecute(Void aVoid) {  
            super.onPostExecute(aVoid);  
            Toast.makeText(MainActivity.this, produto.getNome() + " salvo!",  
                Toast.LENGTH_SHORT).show();  
        }  
    }.execute();  
}
```



```
private void carregarCardapio() {  
  
    progressBar.setVisibility(View.VISIBLE);  
  
    recyclerView.setVisibility(View.GONE);  
  
  
  
    JSONObjectRequest jsonObjectRequest = new JSONObjectRequest(  
  
        Request.Method.GET, URL, null,  
  
        response -> {  
  
            progressBar.setVisibility(View.GONE);  
  
            recyclerView.setVisibility(View.VISIBLE);  
  
            try {  
  
                String productsJson = response.getJSONArray("products").toString();  
  
                Type listType = new TypeToken<List<Produto>>() {}.getType();  
  
                List<Produto> produtos = new Gson().fromJson(productsJson, listType);  
  
                produtoAdapter.setProdutos(produtos);  
  
            } catch (JSONException e) {  
  
                Toast.makeText(MainActivity.this, "Erro ao parsear o JSON",  
                    Toast.LENGTH_LONG).show();  
  
            }  
  
        },  
  
        error -> {  
  
            progressBar.setVisibility(View.GONE);  
  
            Toast.makeText(MainActivity.this, "Falha na comunicação com a API: " +  
                error.getMessage(), Toast.LENGTH_LONG).show();  
  
        }  
  
    );  
}
```

```
        requestQueue.add(jsonObjectRequest);
    }
}

// 5. AnalyticsActivity.java (Activity de Análise - Gráficos)

package com.example.projeto_entrega2;

import android.graphics.Color;

import android.os.Bundle;

import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

import com.android.volley.Request;
import com.android.volley.RequestQueue;
import com.android.volley.toolbox.JsonObjectRequest;
import com.android.volley.toolbox.Volley;

import com.example.projeto_entrega2.model.Produto;

import com.github.mikephil.charting.charts.BarChart;
import com.github.mikephil.charting.charts.PieChart;
import com.github.mikephil.charting.components.XAxis;
import com.github.mikephil.charting.data.BarData;
import com.github.mikephil.charting.data.BarDataSet;
import com.github.mikephil.charting.data.BarEntry;
```

```
import com.github.mikephil.charting.data.PieData;

import com.github.mikephil.charting.data.PieDataSet;

import com.github.mikephil.charting.data.PieEntry;

import com.github.mikephil.charting.formatter.IndexAxisValueFormatter;

import com.github.mikephil.charting.utils.ColorTemplate;

import com.google.gson.Gson;

import com.google.gson.reflect.TypeToken;


import org.json.JSONException;


import java.lang.reflect.Type;

import java.util.ArrayList;

import java.util.HashMap;

import java.util.List;

import java.util.Map;


public class AnalyticsActivity extends AppCompatActivity {


    private PieChart pieChart;

    private BarChart barChart;

    private RequestQueue requestQueue;

    private static final String URL = "https://dummyjson.com/products?limit=10";


    @Override
```



```
protected void onCreate(Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);  
    setContentView(R.layout.activity_analytics);  
  
    pieChart = findViewById(R.id.pieChart);  
    barChart = findViewById(R.id.barChart);  
    requestQueue = Volley.newRequestQueue(this);  
  
    loadProductsAndCreateCharts();  
}  
  
private void loadProductsAndCreateCharts() {  
    JSONObjectRequest jsonObjectRequest = new JSONObjectRequest(  
        Request.Method.GET, URL, null,  
        response -> {  
            try {  
                String productsJson = response.getJSONArray("products").toString();  
                Type listType = new TypeToken<List<Produto>>() {}.getType();  
                List<Produto> produtos = new Gson().fromJson(productsJson, listType);  
                processDataForCharts(produtos);  
            } catch (JSONException e) {  
                Toast.makeText(AnalyticsActivity.this, "Error parsing JSON",  
                    Toast.LENGTH_LONG).show();  
            }  
        },  
    ),
```

```
error -> {  
    Toast.makeText(AnalyticsActivity.this, "API communication failed: " +  
error.getMessage(), Toast.LENGTH_LONG).show();  
  
}  
  
);  
  
requestQueue.add(jsonObjectRequest);  
}  
  
private void processDataForCharts(List<Produto> produtos) {  
  
    // Estruturas para armazenar dados agregados por marca  
  
    Map<String, Integer> brandProductCount = new HashMap<>();  
  
    Map<String, Double> brandTotalPrice = new HashMap<>();  
  
    for (Produto p : produtos) {  
  
        // O campo 'brand' precisa existir no seu GSON/Produto.java  
  
        String brand = p.getMarca() != null ? p.getMarca() : "Unknown";  
  
  
  
        // Contagem de produtos por marca  
  
        brandProductCount.put(brand, brandProductCount.getOrDefault(brand, 0) + 1);  
  
  
  
        // Soma dos preços por marca  
  
        brandTotalPrice.put(brand, brandTotalPrice.getOrDefault(brand, 0.0) + p.getPreco());  
  
    }  
  
    setupPieChart(brandProductCount);  
}
```

```
        setupBarChart(brandProductCount, brandTotalPrice);  
    }  
}
```

```
private void setupPieChart(Map<String, Integer> brandProductCount) {  
  
    List<PieEntry> pieEntries = new ArrayList<>();  
  
    for (Map.Entry<String, Integer> entry : brandProductCount.entrySet()) {  
  
        pieEntries.add(new PieEntry(entry.getValue(), entry.getKey()));  
  
    }  
}
```

```
PieDataSet pieDataSet = new PieDataSet(pieEntries, "");  
  
pieDataSet.setColors(ColorTemplate.MATERIAL_COLORS);  
  
pieDataSet.setValueTextColor(Color.BLACK);  
  
pieDataSet.setValueTextSize(12f);
```

```
PieData pieData = new PieData(pieDataSet);
```

```
pieChart.setData(pieData);  
  
pieChart.getDescription().setEnabled(false);  
  
pieChart.setCenterText("Produtos por Marca");  
  
pieChart.animateY(1000);  
  
pieChart.invalidate();  
  
}
```

```
private void setupBarChart(Map<String, Integer> brandProductCount, Map<String, Double>  
brandTotalPrice) {
```



```
List<BarEntry> barEntries = new ArrayList<>();

List<String> labels = new ArrayList<>();

int index = 0;

for (Map.Entry<String, Double> entry : brandTotalPrice.entrySet()) {

    String brand = entry.getKey();

    double total = entry.getValue();

    int count = brandProductCount.getOrDefault(brand, 1); // Evita divisão por zero

    float avgPrice = (float) (total / count);

    barEntries.add(new BarEntry(index, avgPrice));

    labels.add(brand);

    index++;

}

BarDataSet barDataSet = new BarDataSet(barEntries, "Preço Médio ($)");

barDataSet.setColors(ColorTemplate.MATERIAL_COLORS);

BarData barData = new BarData(barDataSet);

barData.setBarWidth(0.6f);

barChart.setData(barData);

barChart.getDescription().setEnabled(false);

barChart.setFitBars(true);
```

```
XAxis xAxis = barChart.getXAxis();

xAxis.setValueFormatter(new IndexAxisValueFormatter(labels));

xAxis.setPosition(XAxis.XAxisPosition.BOTTOM);

xAxis.setGranularity(1f);

xAxis.setGranularityEnabled(true);

xAxis.setLabelRotationAngle(-45);


barChart.invalidate(); // refresh

}

}
```

// 6. DadosSalvosActivity.java (Activity de Produtos Salvos) package com.example.projeto_entrega2;

```
import android.os.AsyncTask; import android.os.Bundle; import android.view.Menu; import
android.view.MenuInflater; import android.view.MenuItem; import android.view.View; import
android.widget.ProgressBar; import android.widget.Toast;
```

```
import androidx.annotation.NonNull; import androidx.appcompat.app.AppCompatActivity; import
androidx.recyclerview.widget.LinearLayoutManager; import
androidx.recyclerview.widget.RecyclerView;
```

```
import com.example.projeto_entrega2.adapter.ProdutoAdapter; import
com.example.projeto_entrega2.database.AppDatabase; import
com.example.projeto_entrega2.model.Produto;
```

```
import java.util.ArrayList; import java.util.List;
```

```
public class DadosSalvosActivity extends AppCompatActivity {
```

```
private RecyclerView recyclerView;
private ProdutoAdapter produtoAdapter;
private ProgressBar progressBar;
private AppDatabase db;
```

```
@Override
protected void onCreate(Bundle savedInstanceState) {
```

```
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_dados_salvos);

recyclerView = findViewById(R.id.recyclerView);
progressBar = findViewById(R.id.progressBar);

db = AppDatabase.getDatabase(getApplicationContext());

setupRecyclerView();
buscarDadosDoBanco();
}

private void setupRecyclerView() {
    recyclerView.setLayoutManager(new LinearLayoutManager(this));
    produtoAdapter = new ProdutoAdapter();
    recyclerView.setAdapter(produtoAdapter);
}

@Override
public boolean onCreateOptionsMenu(Menu menu) {
    MenuInflater inflater = getMenuInflater();
    inflater.inflate(R.menu.dados_salvos_menu, menu);
    return true;
}

@Override
public boolean onOptionsItemSelected(@NonNull MenuItem item) {
    if (item.getItemId() == R.id.action_delete_all) {
        deleteAllData();
        return true;
    }
    return super.onOptionsItemSelected(item);
}

private void deleteAllData() {
    new AsyncTask<Void, Void, Void>() {
        @Override
        protected Void doInBackground(Void... voids) {
            db.produtoDAO().deleteAll();
            return null;
        }

        @Override
        protected void onPostExecute(Void aVoid) {
```



```
        super.onPostExecute(aVoid);
        produtoAdapter.setProdutos(new ArrayList<>()); // Limpa a lista
na UI
        Toast.makeText(DadosSalvosActivity.this, "Dados apagados!",
Toast.LENGTH_SHORT).show();
    }
    }.execute();
}

private void buscarDadosDoBanco() {
    progressBar.setVisibility(View.VISIBLE);
    new SelectFromDbAsyncTask(this).execute();
}

private static class SelectFromDbAsyncTask extends AsyncTask<Void, Void,
List<Produto>> {
    private final DadosSalvosActivity activity;

    SelectFromDbAsyncTask(DadosSalvosActivity activity) {
        this.activity = activity;
    }

    @Override
    protected List<Produto> doInBackground(Void... voids) {
        return activity.db.produtoDAO().getAll();
    }

    @Override
    protected void onPostExecute(List<Produto> produtos) {
        if (activity != null) {
            activity.progressBar.setVisibility(View.GONE);
            activity.produtoAdapter.setProdutos(produtos);
        }
    }
}

// 7. SplashActivity.java (Activity Inicial/Navegação)

package com.example.projeto_entrega2;
```

```
import android.content.Intent;
```

```
import android.os.Bundle;
```

```
import android.widget.Button;
```

```
import androidx.appcompat.app.AppCompatActivity;
```

```
public class SplashActivity extends AppCompatActivity {
```

```
    @Override
```

```
    protected void onCreate(Bundle savedInstanceState) {
```

```
        super.onCreate(savedInstanceState);
```

```
        setContentView(R.layout.activity_splash);
```

```
        Button btnApiProducts = findViewById(R.id.btnApiProducts);
```

```
        Button btnSavedProducts = findViewById(R.id.btnSavedProducts);
```

```
        Button btnAnalytics = findViewById(R.id.btnAnalytics);
```

```
        btnApiProducts.setOnClickListener(v -> {
```

```
            Intent intent = new Intent(SplashActivity.this, MainActivity.class);
```

```
            startActivity(intent);
```

```
        });
```

```
        btnSavedProducts.setOnClickListener(v -> {
```

```
            Intent intent = new Intent(SplashActivity.this, DadosSalvosActivity.class);
```

```
startActivity(intent);  
});
```

```
btnAnalytics.setOnClickListener(v -> {  
  
    Intent intent = new Intent(SplashActivity.this, AnalyticsActivity.class);  
  
    startActivity(intent);  
  
    });  
}  
}
```

// 8. ProductResponse.java (Modelo de Resposta da API)

```
package com.example.projeto_entrega2.model;
```

```
import java.util.List;
```

```
// Esta classe é um wrapper para a resposta da API DummyJSON.
```

```
// A API retorna um objeto JSON que contém uma lista de produtos sob a chave "products".
```

```
public class ProductResponse {  
  
    private List<Produto> products;  
  
    public List<Produto> getProducts() {  
  
        return products;  
  
    }  
}
```



```
public void setProducts(List<Produto> products) {  
    this.products = products;  
}  
}
```

// 9. Produto.java (Modelo de Dados/Room Entity) package com.example.projeto_entrega2.model;

```
import android.os.Parcel; import android.os.Parcelable; import androidx.room.Entity; import  
androidx.room.PrimaryKey; import com.google.gson.annotations.SerializedName;
```

```
@Entity(tableName = "tabela_de_produtos") public class Produto implements Parcelable {
```

```
@PrimaryKey(autoGenerate = true)  
@SerializedName("id")  
private int id;
```

```
@SerializedName("title")  
private String nome;
```

```
@SerializedName("price")  
private double preco;
```

```
@SerializedName("description")  
private String descricao;
```

```
@SerializedName("category")  
private String categoria;
```

```
@SerializedName("brand") // NOVO CAMPO  
private String marca;
```

```
@SerializedName("thumbnail")  
private String urlImagem;
```

```
// CONSTRUTOR ATUALIZADO
```

```
public Produto(String nome, double preco, String descricao, String  
categoria, String marca, String urlImagem) {  
    this.nome = nome;  
    this.preco = preco;  
    this.descricao = descricao;  
    this.categoria = categoria;  
    this.marca = marca;
```

```
        this.urlImagem = urlImagem;
    }

    // --- Getters ---
    public int getId() { return id; }
    public String getNome() { return nome; }
    public double getPreco() { return preco; }
    public String getDescricao() { return descricao; }
    public String getCategoria() { return categoria; }
    public String getMarca() { return marca; } // NOVO GETTER
    public String getUrlImagem() { return urlImagem; }

    // --- Setter para o ID ---
    public void setId(int id) {
        this.id = id;
    }

    // PARCELABLE ATUALIZADO
    protected Produto(Parcel in) {
        id = in.readInt();
        nome = in.readString();
        preco = in.readDouble();
        descricao = in.readString();
        categoria = in.readString();
        marca = in.readString(); // NOVO CAMPO
        urlImagem = in.readString();
    }

    @Override
    public void writeToParcel(Parcel dest, int flags) {
        dest.writeInt(id);
        dest.writeString(nome);
        dest.writeDouble(preco);
        dest.writeString(descricao);
        dest.writeString(categoria);
        dest.writeString(marca); // NOVO CAMPO
        dest.writeString(urlImagem);
    }

    @Override
    public int describeContents() {
        return 0;
    }
}
```

```
public static final Creator<Produto> CREATOR = new Creator<Produto>() {  
    @Override  
    public Produto createFromParcel(Parcel in) {  
        return new Produto(in);  
    }  
  
    @Override  
    public Produto[] newArray(int size) {  
        return new Produto[size];  
    }  
};  
  
}
```

4. REFERÊNCIAS BIBLIOGRÁFICAS

SOMMERVILLE, I. **Engenharia de Software**. 11ª Edição. São Paulo: Pearson Addison-Wesley, 2017.